Solidity Contract Explanation

```
// 1. We tell the compiler what license we're using. It's just a good habit.
// SPDX-License-Identifier: MIT
// 2. We tell the compiler which version of Solidity to use.
pragma solidity ^0.8.20;
// 3. We define our contract. Think of it like a 'class' in C++.
contract HelloWorld {
    // 4. This is a "State Variable". It is data stored permanently
    //    with the contract on the blockchain.
    string public greeting = "Hello, World!";
    uint myNumber = 30;//uint = unsigned integer (positive whole numbers) which consume total
256 bit
}  Explain this code in details and also make notes related to this
```

Alright! Let's **explain your Solidity code step by step**, and I'll also write **concise notes** you can keep for revision.

✅ Detailed Explanation

`// 1. We tell the compiler what license we're using. It's just a good habit.`

```
solidity

// SPDX-License-Identifier: MIT
```

- **What it does:**
  This line declares the **software license** for your smart contract code.
- **SPDX:** Software Package Data Exchange — a standard way to declare licenses.
- **MIT License:** A very permissive open-source license that lets anyone reuse, modify, and distribute your code with minimal restrictions.
- **Why it's good:**
  Including this helps avoid copyright or legal ambiguity when your contract is published on platforms like **Etherscan**.

`// 2. We tell the compiler which version of Solidity to use.`

```
solidity

pragma solidity ^0.8.20;
```

- `pragma` **keyword:**
  It tells the Solidity compiler which version of the language your code works with.
- `^0.8.20`**:**
  This means: "Use version 0.8.20 or **any newer version** that does not break backward compatibility."
- **Why versioning matters:**
  Solidity evolves often, adding features and fixing bugs — so specifying the version avoids unexpected behavior due to breaking changes.

`// 3. We define our contract. Think of it like a 'class' in C++.`

```
solidity

contract HelloWorld {
```

- `contract` **keyword:**
  This defines a smart contract. A contract is like a **class** in object-oriented languages.
- `HelloWorld`**:**
  This is the name of the contract.
- **What it does:**
  The contract groups together **state variables**, **functions**, and **logic** that can interact with the Ethereum blockchain.

`// 4. This is a "State Variable". It is data stored permanently with the contract on the blockchain.`

```
solidity

string public greeting = "Hello, World!";
```

- **State Variable:**
  `greeting` is stored **on the blockchain** permanently.
- `string` **type:**
  Holds a text value (**`"Hello, World!"`**).
- `public` **keyword:**
  Makes the variable **readable by anyone**. Solidity automatically generates a **getter function** for `public` variables.
- **Where stored:**
  Its value lives inside the blockchain storage, which costs gas to write to.

`uint myNumber = 30;`

```
solidity

uint myNumber = 30;
```

- **`uint`:**
  Short for **`uint256`**. It's an **unsigned integer**, so it can only hold **non-negative whole numbers**.
- **Default bit size:**
  If you write just **`uint`**, it defaults to **`uint256`** (256 bits).
- **Value:**
  `myNumber` is set to **30**.
- **Visibility:**
  No **`public`** keyword means it's **private** by default — it cannot be read directly from outside the contract.
- **Why `unsigned` matters:**
  Negative numbers aren't allowed, which simplifies arithmetic and prevents some bugs.

📌 Key Points to Remember

Concept	Details
SPDX License	Declares the code license (MIT in this case)
Pragma	Compiler version directive
Contract	Like a class in OOP, contains state and logic
State Variable	Stored on-chain permanently
string	Data type for text
uint / uint256	Unsigned integer, 256 bits
public	Creates a getter, visible externally
private	No keyword → default is internal/private

✅ Quick Notes (for Revision)

📌 Solidity Basics

- **`SPDX-License-Identifier`** → Declares open-source license
- **`pragma solidity`** → Sets compiler version range
- **`contract`** → Basic building block, like a class
- **State Variables:** Stored permanently, cost gas
- **`string`** → For text
- **`uint`** → Unsigned integers, default 256 bits
- **`public`** → Generates automatic getter
- No visibility → Defaults to internal

🔑 Why This is Useful

This simple contract shows:

- How to store data on-chain.
- How to make variables accessible to anyone.
- How Solidity code is structured: license → compiler version → contract → variables/functions.

If you'd like, I can also explain:

- How to deploy this contract.
- How to interact with it (e.g., read `greeting`).
- How to add functions to change `greeting` or `myNumber`.

Want me to do that too? 🚀✨